

approximate measurement of the chip temperature. Although it is not intended for precise ambient temperature measurement, it is useful for monitoring temperature trends and detecting overheating during operation. Table 4 lists the measured chip temperature ranges.

The `getChipTempAvg()` function samples the internal temperature sensor multiple times, averages the readings to minimise fluctuations, and returns a more stable estimate of the chip temperature. Fig. 6 shows the `getChipTempAvg()` function used to calculate the average on-chip temperature of the ESP32-C3.

Flash usage (%)

Flash usage represents the size of the compiled firmware relative to the total available flash memory of the ESP32-C3. The flash usage percentage is calculated as:

$$\text{Flash usage (\%)} = \left(\frac{\text{Used flash}}{\text{total flash}} \right) \times 100$$

The `getFlashUsedPercent()` function reads the total flash chip size and the compiled sketch size before calculating the percentage of flash memory occupied by the firmware. Monitoring flash usage helps ensure that sufficient flash memory remains available for future firmware updates and additional application features. Fig. 7 shows the function used to calculate ESP32-C3 firmware flash usage.

Reset reason

The reset reason indicates why the ESP32-C3 restarted. Monitoring this information helps identify whether the reset was caused by normal operation or an unexpected event. Displaying the reset reason on the IoT dashboard enables remote diagnosis of system failures and improves troubleshooting efficiency. Table 5 lists the common ESP32-C3 reset causes, while Fig. 8 shows the function used to identify and return the reset reason.

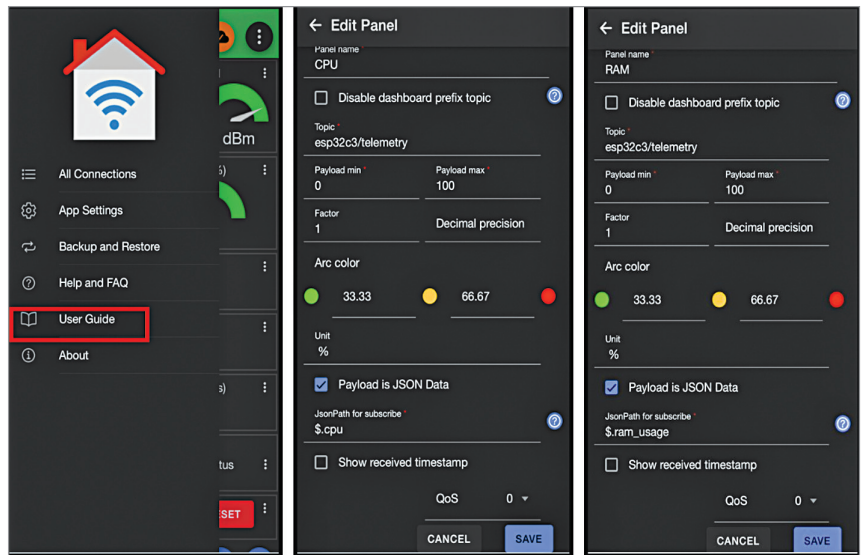


Fig. 14: Accessing the IoT MQTT panel user guide and configuring the CPU and RAM telemetry panels

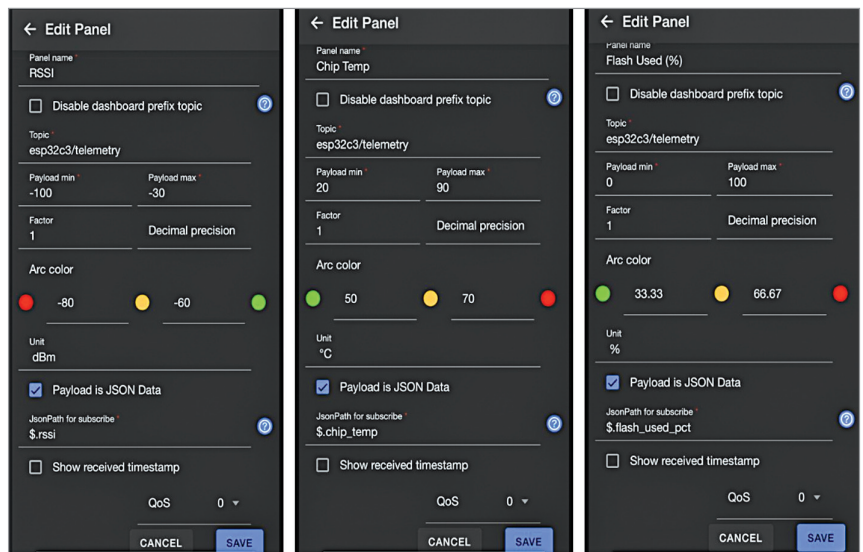


Fig. 15: Configuration of the RSSI, chip temperature, and flash usage telemetry panels in the IoT MQTT panel app

Wi-Fi channel (CHx)

The Wi-Fi channel indicates the frequency channel used by the wireless access point to which the ESP32-C3 Super Mini is connected. For 2.4GHz Wi-Fi networks, the channel number typically ranges from 1 to 13 (or 1 to 14, depending on the region). Monitoring the Wi-Fi channel helps identify network congestion, interference from nearby Wi-Fi networks, and channel overlap that may affect communication performance. Fig. 9 shows the function used to retrieve the connected Wi-Fi channel.

Uptime (Days: Hours: Minutes: Seconds)

Uptime indicates how long the ESP32-C3 has been running continuously since its last reset. It is calculated using the internal system timer (`millis()`), which measures the elapsed time since the microcontroller was powered on or restarted. The uptime is formatted as `DDD:HH:MM:SS` (Days:Hours:Minutes:Seconds).

Example: 002:05:14:22 (2 days, 5 hours, 14 minutes, and 22 seconds)

Monitoring uptime helps verify